Backpropagation and Gradient Descent

This handout introduces the backpropagation and gradient descent algorithms used to train neural networks, with particular reference to their implementation in the Inside the Network application.

1. Introduction

During neural-network training, weights and biases are adjusted to reduce the loss E. For a generic parameter p, gradient descent uses the derivative $\partial E / \partial p$ to determine the update:

$$p_{\text{new}} = p - \eta \frac{\partial E}{\partial p}, \tag{1}$$

where the derivative is evaluated at the current value p of the parameter; η is the learning rate. In *feed-forward* networks, where connections run from the inputs toward successive layers without forming cycles, the standard method for computing these derivatives is *backpropagation*. As we will see, it applies the chain rule from the output layer back through the preceding layers and, through a recursive formulation, reuses derivatives that have already been computed.

The same procedure is made visible in the app's **One Data Point at a Time** mode. We will first develop the calculation in index notation, then connect it to training over the full dataset, and finally collect the results in vector and matrix form.

Notation and meaning of the symbols

We number the layers from 0 to L. Layer 0 contains the inputs; layer L is the output layer. Let n_ℓ denote the number of nodes in layer ℓ; the nodes in layer ℓ are numbered from 1 to n_ℓ.

Let $x = \left(x_1, \dots, x_{n_0}\right)^T$ be the column vector containing the inputs for a single example, and set $a_i^{(0)} = x_i$. We will use the following notation:

Symbol	Meaning
$a_i^{(\ell)}$	value of node i in layer ℓ; for $\ell = 0$, $a_i^{(0)} = x_i$
$w_{ij}^{(\ell)}$	weight of the connection from node j in layer $\ell - 1$ to neuron i in layer ℓ
$b_i^{(\ell)}$	bias of neuron i in layer ℓ
$z_i^{(\ell)}$	pre-activation of neuron i in layer ℓ
f_ℓ	activation function used in layer ℓ
E	loss for the current example
$\delta_i^{(\ell)}$	derivative $\partial E / \partial z_i^{(\ell)}$
η	learning rate

This material is still under review and may contain inaccuracies or errors. Comments and corrections are welcome. Last updated: September 2, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

In equation (1), p therefore denotes the current value of any weight $w_{ij}^{(\ell)}$ or bias $b_i^{(\ell)}$ in the network.

For $a_i^{(\ell)}$, $\ell = 0$ is also allowed; for $w_{ij}^{(\ell)}$, $b_i^{(\ell)}$, $z_i^{(\ell)}$, f_ℓ, and $\delta_i^{(\ell)}$, we assume $\ell = 1, \dots, L$. Thus, whenever we consider the transition from layer $\ell - 1$ to layer ℓ, we have $\ell = 1, \dots, L$.

In single-output problems, y denotes the target value; in the multiclass case, c is the index of the correct class and t_i are the components of the *one-hot* target vector (1 for the correct class, 0 for all others).

Connection to the app code

The array structure used in the core of the app maps directly to the mathematical notation. However, neuron indices in this handout start at 1, whereas JavaScript array indices start at 0; therefore the table uses i−1 and j−1 for neuron indices in the arrays.

Mathematical notation	Array in the app
$w_{ij}^{(\ell)}$	weights[ℓ−1][i−1][j−1]
$b_i^{(\ell)}$	bias[ℓ−1][i−1]
$z_i^{(\ell)}$	zs[ℓ][i−1]
$a_i^{(\ell)}$	as[ℓ][i−1]
$\delta_i^{(\ell)}$	delta[ℓ][i−1]

In particular, as[0] contains the inputs; zs[ℓ] and delta[ℓ] correspond to layers $\ell = 1, \dots, L$. The arrays weights[ℓ−1] and bias[ℓ−1] instead represent the transition from layer $\ell - 1$ to layer ℓ.

2. Forward pass: computing the output in index notation

For $\ell = 1, \dots, L$, neuron i in layer ℓ receives the values $a_j^{(\ell-1)}$ from all nodes in the preceding layer and first computes the weighted sum

$$z_i^{(\ell)} = \sum_{j=1}^{n_{\ell-1}} w_{ij}^{(\ell)} \, a_j^{(\ell-1)} + b_i^{(\ell)} \tag{2}$$

For layers with an elementwise activation, we then have:

$$a_i^{(\ell)} = f_\ell\big(z_i^{(\ell)}\big). \tag{3}$$

In the app's multiclass case, the output layer instead applies softmax to the entire pre-activation vector. The *forward pass*, that is, computing the network output from the inputs, repeats these operations layer by layer until the output is reached. For a single-output problem, the network output is

$$\hat{y} = a_1^{(L)}.$$

For binary classification and regression, the app uses the squared-error loss for each example

$$E = \frac{1}{2}(\hat{y} - y)^2.$$

For multiclass classification, the output is a probability vector produced by softmax and the loss is cross-entropy

$$E = -\ln p_c.$$

3. The quantity δ

As seen in equation (1), updating weights and biases by gradient descent requires the partial derivatives

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} \quad \text{e} \quad \frac{\partial E}{\partial b_i^{(\ell)}}.$$

To organize this computation, we associate with each neuron a quantity that will be reused during the *backward pass*:

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

The symbol $\delta_i^{(\ell)}$, often informally called the **error signal**, is the derivative of the loss E with respect to the corresponding neuron's pre-activation $z_i^{(\ell)}$; it therefore expresses the local, signed sensitivity of the loss to changes in the pre-activation.

When the activation is applied elementwise, since

$$a_i^{(\ell)} = f_\ell\big(z_i^{(\ell)}\big),$$

the chain rule also gives

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial a_i^{(\ell)}}\, f_\ell'\big(z_i^{(\ell)}\big).$$

In the next section we will use $\delta_i^{(\ell)}$ to obtain the derivatives with respect to the weights and bias feeding into the neuron.

4. Derivatives with respect to a weight and a bias

Consider a specific weight, $w_{ij}^{(\ell)}$. In the sum

$$z_i^{(\ell)} = \sum_{q=1}^{n_{\ell-1}} w_{iq}^{(\ell)}\, a_q^{(\ell-1)} + b_i^{(\ell)},$$

this weight appears only in the term $w_{ij}^{(\ell)} a_j^{(\ell-1)}$. Therefore

$$\frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = a_j^{(\ell-1)}.$$

By the chain rule:

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \frac{\partial E}{\partial z_i^{(\ell)}} \frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}. \tag{4}$$

For the bias,

$$\frac{\partial z_i^{(\ell)}}{\partial b_i^{(\ell)}} = 1,$$

and therefore

$$\frac{\partial E}{\partial b_i^{(\ell)}} = \delta_i^{(\ell)}. \tag{5}$$

The two relations can therefore be summarized as:

$$\frac{\partial E}{\partial w} = \delta_{\text{destination}}\, a_{\text{source}}, \qquad \frac{\partial E}{\partial b} = \delta_{\text{neuron}}.$$

5. The quantity δ in the output layer

For output neurons, the values of $\delta_i^{(L)}$ can be derived directly from the loss function and the output transformation. This section works through the cases implemented in the app. In hidden layers, by contrast, the values of $\delta_i^{(\ell)}$ are computed recursively from those in the next layer, as described in §6.

5.1 Binary classification in the app: sigmoid + squared-error loss

With a single output, using a sigmoid and squared-error loss as in the app, we have

$$a = \hat{y} = \sigma(z), \qquad E = \frac{1}{2}(a - y)^2.$$

Since

$$\frac{\partial E}{\partial a} = a - y \qquad \text{e} \qquad \sigma'(z) = a(1 - a),$$

hence

$$\delta^{(L)} = (a - y)\, a(1 - a). \tag{6a}$$

In practice, however, binary classification commonly pairs a sigmoid with binary cross-entropy; in that case the output delta simplifies to $\delta = \hat{y} - y$.

5.2 Regression in the app: linear output + squared-error loss

If the output layer uses $a = z$, then $f'(z) = 1$, and hence

$$\delta^{(L)} = a - y\,. \tag{6b}$$

5.3 Multiclass classification: softmax + cross-entropy

If $z_i^{(L)}$ is the pre-activation of output neuron i, softmax gives

$$p_i = \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}}.$$

We use one-hot encoding: t_i is the target component, with $t_i = 1$ for the correct class and $t_i = 0$ for all others. The cross-entropy loss for a single example is

$$E = -\sum_{q=1}^{n_L} t_q \ln p_q = -\ln p_c,$$

where c is the correct class. Since

$$\ln p_q = z_q^{(L)} - \ln\left(\sum_{k=1}^{n_L} e^{z_k^{(L)}}\right)$$

and $\sum_i t_i = 1$, we can rewrite this as

$$E = -\sum_{q=1}^{n_L} t_q z_q^{(L)} + \ln\left(\sum_{k=1}^{n_L} e^{z_k^{(L)}}\right).$$

Differentiating with respect to $z_i^{(L)}$ gives

$$\delta_i^{(L)} = \frac{\partial E}{\partial z_i^{(L)}} = -t_i + \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}} = p_i - t_i. \tag{6c}$$

6. The quantity δ in a hidden layer

Consider neuron i in a hidden layer ℓ. Its activation $a_i^{(\ell)}$ reaches every neuron $k = 1, \dots, n_{\ell+1}$ in the next layer. For each of them,

$$z_k^{(\ell+1)} = \sum_{q=1}^{n_\ell} w_{kq}^{(\ell+1)} a_q^{(\ell)} + b_k^{(\ell+1)}.$$

We want to compute

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

The loss depends on $z_i^{(\ell)}$ through the activation $a_i^{(\ell)}$ and then through all neurons in the next layer. The chain rule must therefore sum over all these paths:

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} \frac{\partial E}{\partial z_k^{(\ell+1)}} \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}}.$$

But

$$\frac{\partial E}{\partial z_k^{(\ell+1)}} = \delta_k^{(\ell+1)} \qquad \text{e} \qquad \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}} = w_{ki}^{(\ell+1)},$$

therefore

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}.$$

Finally, $a_i^{(\ell)} = f_\ell\left(z_i^{(\ell)}\right)$, so

$$\delta_i^{(\ell)} = f_\ell{}'\left(z_i^{(\ell)}\right) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}. \tag{7}$$

This recursive formula therefore computes the term δ in a hidden layer from the terms δ in the next layer.

In mnemonic form, the recurrence has the following structure:

$$\delta_{\text{neuron}} = f'(z_{\text{neuron}}) \cdot \sum_{\text{next}} (w \cdot \delta_{\text{next}}).$$

7. Backpropagation for a single example

For one example, with the inputs and target fixed:

1. **Forward.** Compute and store all $z_i^{(\ell)}$ and $a_i^{(\ell)}$ values using (2) and (3).
2. **Loss.** Compute E from the network output and the target.
3. **Output delta.** Compute the $\delta_i^{(L)}$ values from the loss function and the output transformation. For the cases implemented in the app, the appropriate expression is (6a), (6b), or (6c).
4. **Hidden deltas.** Compute the $\delta_i^{(\ell)}$ values from the last hidden layer back to the first by applying (7) recursively.
5. **Parameter derivatives.** Once all $\delta_i^{(\ell)}$ terms are known, the derivatives of E with respect to the weights and biases follow from (4) and (5).
6. **Update.** Compute the parameter changes using (1) and apply them together.

All derivatives must refer to the same network state: first compute the derivatives of E with respect to every weight and bias, and only then apply the updates simultaneously.

8. From a single example to the dataset: SGD and mini-batches

So far we have computed the derivatives of the loss $E_r(\theta)$ for a single example. Training, however, aims to minimize the mean loss over the full set of N training examples. Let θ collect all weights and biases in the network, and let $\bar{E}(\theta)$ denote the mean loss:

$$\bar{E}(\theta) = \frac{1}{N} \sum_{r=1}^{N} E_r\,(\theta).$$

By linearity of differentiation, the gradient of the mean loss with respect to the parameters is the mean of the gradients $\nabla E_r(\theta)$ of the losses associated with the individual examples:

$$\nabla \bar{E}(\theta) = \frac{1}{N} \sum_{r=1}^{N} \nabla E_r(\theta).$$

Gradient descent over the full dataset. With the parameters held fixed at their current values, compute the gradients of the losses associated with all examples in the dataset, average them, and apply a single update:

$$\theta_{\text{new}} = \theta - \eta\, \nabla \bar{E}(\theta).$$

This procedure is called batch gradient descent.

Stochastic gradient descent (SGD). It is stochastic because, at each step, the update direction depends on a randomly selected example rather than being computed from the entire dataset at once. Under independent sampling, let R denote the index of the example selected at a given step. At each step, R is drawn uniformly and independently from $1, \ldots, N$, and the parameters are updated immediately:

$$\theta_{\text{new}} = \theta - \eta\, \nabla E_R(\theta).$$

Because each index has probability $1/N$, with the parameters fixed the expected gradient $\nabla E_R(\theta)$ is:

$$\mathbb{E}_R[\nabla E_R(\theta)] = \sum_{r=1}^{N} \frac{1}{N} \nabla E_r(\theta) = \frac{1}{N} \sum_{r=1}^{N} \nabla E_r(\theta) = \nabla \bar{E}(\theta).$$

The expectation here is taken with respect to the random choice of R among the N examples in the fixed dataset. In this sense, the gradient of the loss for the sampled example is an unbiased estimator of the gradient of the mean loss. Consequently, with the parameters fixed, the expected update of one SGD step matches the update of one batch gradient descent step with the same learning rate η, even though any individual update may differ.

Random reshuffling. At the beginning of each epoch, a new random ordering $r_1, \dots, r_N$ of the examples is generated, and every example is then used exactly once; sampling is therefore without replacement. This is the algorithm used by the app.

Random reshuffling, meaning that the dataset is shuffled at each epoch and then processed without replacement, is common in practice.

If the parameters were held fixed at their initial value θ_0 throughout the epoch, the average of the gradients would equal the gradient of the mean loss exactly, regardless of the order of the examples:

$$\frac{1}{N} \sum_{k=1}^{N} \nabla E_{r_k}(\theta_0) = \frac{1}{N} \sum_{r=1}^{N} \nabla E_r(\theta_0) = \nabla \bar{E}(\theta_0).$$

Because every permutation contains each example exactly once, under this fixed-parameter assumption the average gradient over the epoch is exactly the mean gradient over the dataset.

In actual random reshuffling with an update after each example, as in the app, the parameters change during the epoch. If θ_k denotes the parameter vector after the k-th update, then

$$\theta_k = \theta_{k-1} - \eta \nabla E_{r_k}(\theta_{k-1}).$$

Define the average gradient actually used over the epoch:

$$g_{\text{epoch}} = \frac{1}{N} \sum_{k=1}^{N} \nabla E_{r_k}(\theta_{k-1}).$$

To compare it with $\nabla \bar{E}(\theta_0)$, first note that after $k-1$ updates,

$$\theta_{k-1} - \theta_0 = -\eta \sum_{s=1}^{k-1} \nabla E_{r_s}(\theta_{s-1}).$$

For a fixed number of examples, if the gradients remain bounded in the neighborhood traversed by the parameters, the sum contains only finitely many terms and therefore

$$\theta_{k-1} - \theta_0 = O(\eta).$$

If, in addition, the gradient of each E_r varies smoothly enough with the parameters, for example if it is locally Lipschitz continuous, an $O(\eta)$ change in the parameters produces an $O(\eta)$ change in the gradient:

$$\nabla E_{r_k}(\theta_{k-1}) = \nabla E_{r_k}(\theta_0) + O(\eta).$$

Averaging over the N updates in the epoch, still with N fixed, gives

$$g_{\text{epoch}} = \frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_0) + O(\eta) = \nabla \bar{E}(\theta_0) + O(\eta).$$

Thus, the average gradient actually used during an epoch differs from the gradient of the mean loss evaluated at the beginning of the epoch by a quantity of order $O(\eta)$.

The total update over the epoch is

$$\theta_N = \theta_0 - N\eta\, g_{\text{epoch}} = \theta_0 - N\eta\, \nabla \bar{E}(\theta_0) + O(\eta^2).$$

The leading parameter displacement is of order η, whereas the difference from the corresponding batch step is of order η^2. Therefore, for sufficiently small η, one full epoch of random reshuffling with an update after each example has, to first order, the same update as one batch gradient descent step on the mean loss with learning rate $N\eta$. The factor N appears because an epoch consists of N updates, each using learning rate η, whereas batch gradient descent uses the mean gradient. Shuffling and the intermediate updates determine the higher-order corrections and therefore the exact trajectory. All updates act on the same parameter vector: each example is processed at the network state left by the preceding updates.

Mini-batches. In practice, a common compromise between batch gradient descent and one-example-at-a-time updates is to use a mini-batch B of m examples, for which we compute

$$g_B(\theta) = \frac{1}{m}\sum_{r\in B} \nabla E_r(\theta)$$

and update the network using $g_B(\theta)$. If B is sampled uniformly with the parameters fixed, $g_B(\theta)$ is an unbiased estimator of the gradient of the mean loss. Because it averages contributions from several examples, this quantity typically varies less from one sample to another than the gradient of the loss for a single example, and therefore provides a less noisy estimate of the full-dataset mean gradient. Compared with the full batch, each update requires less computation and can exploit parallel computation efficiently. In practice, mini-batches are often formed by shuffling the dataset at the beginning of an epoch, partitioning it into groups, and updating the parameters after each group. A one-example update corresponds to $m = 1$; the full batch corresponds to $m = N$. The app uses $m = 1$ together with random reshuffling without replacement within each epoch.

9. From index notation to matrix form

The index formulas can be collected into vectors and matrices by writing the same equations simultaneously for all neurons in a layer.

For a layer $\ell = 1, \dots, L$, define:

• $z^{(\ell)} \in \mathbb{R}^{n_\ell}$ is the column vector of pre-activations.

• $a^{(\ell)} \in \mathbb{R}^{n_\ell}$ is the column vector of activations.

• $b^{(\ell)} \in \mathbb{R}^{n_\ell}$ is the column vector of biases.

• $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ is the weight matrix. Under the convention $\left(W^{(\ell)}\right)_{ij} = w_{ij}^{(\ell)}$, its i-th row contains the weights of the connections entering neuron i in layer ℓ and its j-th column contains the weights of the connections leaving node j in the preceding layer $\ell - 1$ of the network.

Forward pass

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)},$$
$$a^{(\ell)} = f_\ell\left(z^{(\ell)}\right).$$

In the app, f_ℓ is applied elementwise, except for softmax in the output layer.

Backward pass in hidden layers

For a hidden layer ($1 \le \ell \le L-1$), collect the terms $\delta_i^{(\ell)}$ into a column vector:

$$\delta^{(\ell)} = \begin{pmatrix} \delta_1^{(\ell)} \\ \vdots \\ \delta_{n_\ell}^{(\ell)} \end{pmatrix} \in \mathbb{R}^{n_\ell}.$$

For the i-th component, the formula derived in §6 is

$$\delta_i^{(\ell)} = f_\ell'\left(z_i^{(\ell)}\right) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}.$$

The sum on the right is the i-th component of the matrix product $\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}$:

$$\left[\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}\right]_{i,1} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}$$

Writing all components at once gives

$$\delta^{(\ell)} = \left(\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}\right) \odot f_\ell'\left(z^{(\ell)}\right).$$

where $\odot$ denotes the elementwise, or Hadamard, product.

Derivatives of E with respect to the parameters

Let $\frac{\partial E}{\partial W^{(\ell)}}$ denote the matrix $n_\ell \times n_{\ell-1}$ defined by

$$\left[\frac{\partial E}{\partial W^{(\ell)}}\right]_{ij} = \frac{\partial E}{\partial w_{ij}^{(\ell)}}$$

From equation (4),

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}.$$

The right-hand side of equation (4) is the (i,j) entry of the matrix product of the column vector $\delta^{(\ell)}$ and the row vector $\left(a^{(\ell-1)}\right)^T$. We can therefore collect all derivatives of E with respect to the weights in the matrix:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)}\left(a^{(\ell-1)}\right)^T.$$

For the biases, define $\frac{\partial E}{\partial b^{(\ell)}}$ as the column vector with components $\partial E / \partial b_i^{(\ell)}$. Equation (5) then gives the vector identity

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

Extension to mini-batches. Up to this point, $z^{(\ell)}$, $a^{(\ell)}$, and $\delta^{(\ell)}$ are vectors for a single example. With a mini-batch, the vectors corresponding to different examples can be placed side by side as columns of matrices. The forward pass, backward pass, and gradient computation then become largely matrix operations, which are particularly well suited to parallel execution on GPUs.